Applied AI Database Roadmap

Applied AI Engineering: Production Database & Vector Stack Roadmap

Core Stack: FastAPI • Pydantic v2 • PostgreSQL • pgvector • Psycopg 3 • SQLAlchemy 2.0 / SQLModel

---

PART 1: Raw SQL & pgvector (6 Core Production Topics)

Phase 1: Indexing & Runtime Tuning

- **Topic 1: Specialized Index DDL & Operator Classes (For Alembic)**
  - Distance operator classes: `vector_cosine_ops` (`<=>`), `vector_l2_ops` (`<->`), `vector_ip_ops` (`<#>`).
  - HNSW index build parameters (`m`, `ef_construction`) and IVFFlat (`lists`).
- **Topic 2: Runtime Session Tuning (GUC Parameters)**
  - Controlling recall vs. latency per transaction (`SET LOCAL hnsw.ef_search = 100;`, `SET LOCAL ivfflat.probes = 10;`).
  - Provisioning `maintenance_work_mem` for in-memory index builds.

Phase 2: Hybrid Retrieval & Multi-Tenancy

- **Topic 3: Hybrid Search & Reciprocal Rank Fusion (RRF)**
  - Combining dense vector search with sparse PostgreSQL Full-Text Search (`tsvector`, `websearch_to_tsquery`, `ts_rank_cd`).
  - Writing Common Table Expressions (CTEs) and window functions (`ROW_NUMBER() OVER`) to merge scores via RRF.
- **Topic 4: Filtered ANN Searches & Partial Indexes**
  - Scoping vector searches by `tenant_id`, `user_id`, and `is_active`.
  - Building partial HNSW indexes (`WHERE is_active = TRUE`) to prevent index bloat.

Phase 3: Diagnostics & Memory Scale

- **Topic 5: Execution Plan Analysis (`EXPLAIN (ANALYZE, BUFFERS)`)**

    - Debugging `Index Scan` vs. `Seq Scan` fallbacks and verifying memory buffer hits (`shared hit` vs. `read`).
- **Topic 6: Memory Optimization & Vector Quantization**
    - Using 16-bit half-precision vectors (`halfvec`) to cut RAM usage by 50%.
    - Storing learned sparse embeddings (`sparsevec`) and on-the-fly vector casting.

PART 2: Psycopg 3 Driver Layer (10 Core Production Topics)

Phase 1: Core Async Engine & Connection Management

- **Topic 1: `AsyncConnection` & `AsyncCursor`**
    - Non-blocking async query execution loop designed for FastAPI and `asyncio`.
- **Topic 2: `AsyncConnectionPool` (`psycopg_pool`)**
    - Managing pre-warmed connection pools, health checks, timeouts, and FastAPI `lifespan` startup/shutdown.
- **Topic 3: Async Transactions (`conn.transaction()`)**
    - Context-managed atomic blocks, automatic rollbacks on exceptions, and nested savepoints.

Phase 2: Data Typing, Mapping, & `pgvector`

- **Topic 4: Type Adapters & `register_vector_async`**
    - Translating Python lists, NumPy arrays, and `pgvector` embeddings natively without manual string casting.
- **Topic 5: Row Factories (`dict_row` & `class_row`)**
    - Returning query results directly as Python dictionaries or hydrated Pydantic models instead of raw tuples.
- **Topic 6: Binary Protocol (`Format.BINARY`)**
    - Sending and receiving high-dimensional vector embeddings in compact binary format for low CPU overhead.

Phase 3: High-Throughput Ingestion & Dynamic Search

- **Topic 7: Bulk Ingestion via `AsyncCursor.copy()`**
    - Streaming thousands of RAG chunks directly to disk using PostgreSQL's native binary `COPY` protocol.
- **Topic 8: Safe SQL Composition (`psycopg.sql`)**
    - Using `sql.SQL`, `sql.Identifier`, and `sql.Placeholder` to build dynamic multi-tenant queries safely without SQL injection.

Phase 4: Advanced Production & Agentic Patterns

- **Topic 9: Pipeline Mode (`conn.pipeline()`)**
    - Dispatching multiple queries in a single network batch without waiting for sequential round-trip responses.
- **Topic 10: Asynchronous Pub/Sub (`conn.notifies()`)**

- Listening to PostgreSQL `LISTEN / NOTIFY` channels for real-time event triggers and background agent workflows.

PART 3: SQLAlchemy 2.0 & SQLModel (18 Core Production Topics)

Phase 1: Async Engine & Session Architecture (FastAPI Integration)

- **Topic 1: `create_async_engine` & Psycopg 3 Async Dialect**
  - Configuring `postgresql+psycopg://`, pool sizing (`pool_size`, `max_overflow`), and `pool_pre_ping`.
- **Topic 2: `async_sessionmaker` & FastAPI Dependency Injection (`Depends`)**
  - Managing request-scoped `AsyncSession` contexts with `expire_on_commit=False`.
- **Topic 3: Engine Connection Listeners (`@event.listens_for`)**
  - Auto-registering `register_vector_async` on every new raw Psycopg connection checked out by the engine.

Phase 2: Modern Declarative Schema & AI Data Types

- **Topic 4: Modern Declarative Base 2.0 (`Mapped` & `mapped_column`)**
  - Defining strictly typed ORM models, server-side UUID defaults, and timezone-aware timestamps.
- **Topic 5: Native AI Column Types (`Vector`, `JSONB`, `UUID`)**
  - Integrating `pgvector.sqlalchemy.Vector(dim)`, PostgreSQL `JSONB` metadata, and constraints (`CheckConstraint`, `UniqueConstraint`).
- **Topic 6: Pydantic v2 Serialization & Model Separation**
  - Hydrating Pydantic response models cleanly using `model_validate(orm_obj, from_attributes=True)`.

Phase 3: Querying, Vector Retrieval & Raw SQL Hatches

- **Topic 7: Modern 2.0 `select()` & Execution (`scalars()` vs `execute()`)**
  - Type-safe querying, filtering (`where()`, `ilike()`), sorting, and offset pagination (`limit()`, `offset()`).
- **Topic 8: Vector Distance Operators in SQLAlchemy**
  - Executing semantic search in Python via `.cosine_distance()`, `.l2_distance()`, and `.max_inner_product()`.
- **Topic 9: Raw SQL Execution with `text()` & Parameter Binding**
  - Running complex CTEs, Hybrid Search, and Reciprocal Rank Fusion safely inside `session.execute(text(...))`.

Phase 4: Agent Memory, Relationships & Async Loading

- **Topic 10: Relationships & Cascading Deletions (`relationship`)**
  - Linking parent-child hierarchies (`User` -> `ChatSession` -> `ChatMessage`) with `cascade="all, delete-orphan"`.
- **Topic 11: Async Eager Loading Strategies (`selectinload` & `joinedload`)**

    - Preventing async `MissingGreenlet` crashes when fetching conversation history and related agent records.

Phase 5: High-Performance Ingestion & Mutation

- **Topic 12: PostgreSQL Native Upsert (`insert().on_conflict_do_update`)**
    - Using `sqlalchemy.dialects.postgresql.insert` for idempotent RAG chunk ingestion and deduplication.
- **Topic 13: Batch Mutation & Raw Connection Borrowing**
    - Bulk inserting with `returning()` and borrowing `await session.connection()` to run native Psycopg `COPY`.

Phase 6: Production Migrations with Alembic

- **Topic 14: Async Alembic Setup (`env.py`)**
    - Configuring Alembic to run asynchronous migrations against PostgreSQL and auto-detect model changes.
- **Topic 15: Vector Index DDL Migrations**
    - Writing `op.execute()` migrations to enable the `vector` extension and build tuned HNSW indexes.

Phase 7: AI-Specific Production Safeguards

- **Topic 16: LLM Streaming Session Scoping & Background Task Isolation**
    - Releasing DB connections before slow LLM streaming responses or background tasks to prevent connection pool exhaustion.
- **Topic 17: In-Place JSONB Mutation Tracking (`MutableDict.as_mutable(JSONB)`)**
    - Ensuring nested dictionary updates inside agent state or chunk metadata columns are detected and saved on commit.
- **Topic 18: Concurrency Locking for AI Queues & Token Balances (`with_for_update`)**
    - Using `.with_for_update(skip_locked=True)` for parallel embedding workers and atomic credit balance deductions.